

CHAPTER 3

ENUMERATION

Now that an attacker has successfully identified live hosts and running services using the techniques discussed in Chapter 2, he will typically turn next to probing the identified services more fully for known weaknesses, a process we call *enumeration*. It is also worth noting that, as an attacker progresses through later stages of the attack and obtains connectivity to hosts and segments he previously did not have access to, he will often return to this phase to find ways to greatly expand his foothold and work toward specific targets.

The key difference between the previously discussed information-gathering techniques and enumeration is in the level of intrusiveness. Enumeration involves active connections to systems and directed queries. As such, they may (should!) be logged or otherwise noticed. We will show you what to look for and how to block them, if possible.

Much of the information garnered through enumeration may appear harmless at first glance. However, the information that leaks from the following holes can be your undoing, as we illustrate throughout this chapter. In general, the information attackers seek via enumeration includes user account names (to inform subsequent password-guessing attacks), oft-misconfigured shared resources (for example, unsecured file shares), and older software versions with known security vulnerabilities (such as web servers with remote buffer overflows). Once a service is enumerated, it's usually only a matter of time before the intruder compromises the system in question to some degree, if not completely. By closing these easily fixed loopholes, you eliminate the attacker's first foothold.

Enumeration techniques tend to be platform-specific and are, therefore, heavily dependent on information gathered in Chapter 2 (port scans and OS detection). In fact, port scanning and enumeration functionality are often bundled into the same tool, as you saw in Chapter 2 with programs such as SuperScan, which can scan a network for open ports and simultaneously grab banners from any it discovers listening. This chapter will begin with a brief discussion of banner grabbing, the most generic of enumeration techniques, and then delve into more platform-specific mechanisms that may require more specialized tools.

We will discuss services in numeric order, according to the port on which they traditionally listen, whether TCP or UDP—for example, we discuss TCP 21 (FTP) first, TCP 23 (telnet) next, TCP 25 (SMTP) after that, and so on. This chapter does not exhaustively cover every conceivable enumeration technique against all 65,535 TCP and UDP ports; we focus only on those services that have traditionally given up the lion's share of information about target systems, based on our experiences as professional security testers. We hope this more clearly illustrates how enumeration is designed to help provide a more concise understanding of the target, along the way to advancing the attacker's main agenda of unauthorized system access.

NOTE

Throughout this chapter, we use the phrase “NT Family” to refer to all systems based on Microsoft’s “New Technology” (NT) platform, including Window NT 3.x–4.x, Windows 2000, Windows XP, Windows 2003, Windows Vista, Windows 7, and Windows Server 2008. Where necessary, we differentiate between desktop and server versions. In contrast, we refer to the legacy Microsoft DOS/Windows 1.x/3.x/9x/Me lineage as the “DOS Family.”

SERVICE FINGERPRINTING

The bulk of this chapter focuses on manual techniques for enumerating specific services, such as SMTP, DNS, and SNMP. But before we jump into a discussion of those manual techniques, we need to point out automated techniques for evaluating entire networks for the same information—quickly and efficiently—using a process called *service fingerprinting*. Given the power and scale of these techniques, they are most likely to be used by modern attackers, unless extreme stealth is required, in which case manual hunt-and-peck will be employed.

In Chapter 2, we discussed how to scan for open ports across one or more networks. Service fingerprinting goes one step further, revealing the actual services (and deeper information such as their revision/patch level) associated with each port. Service fingerprinting is more thorough and provides more valuable information than scanning, but it is also more time consuming and noticeable because it generates considerably more traffic.



Nmap Version Scanning

Popularity:	9
Simplicity:	8
Impact:	3
Risk Rating:	7

Chapter 2 introduced you to the powerful and free network scanning tool Nmap (nmap.org) and its scanning and operating system identification capabilities. As you may have noticed in the prior discussion, by default, Nmap lists service names along with ports. This service information is obtained from a file named `nmap-services`, which is simply a text file mapping services with their commonly associated ports. Nmap utilized with the `-sV` switch goes a step further and interrogates the ports, soliciting feedback and matching what it receives with known protocols and specific protocol version information using a different file called `nmap-service-probe`, which contains information on known service responses. With this additional insight, you can identify “hidden” services, such as an exploitable OpenSSH 3.7 service running on TCP port 1417 (as opposed to the default SSH port 22), without overlooking it as an otherwise less-interesting Timbuktu server (normally found on port 1417). The following example

Nmap output demonstrates this scenario. First, here's an Nmap SYN scan misidentifying the service:

```
[root$] nmap -sS target.com -p 1417
```

```
Starting Nmap 4.68 ( http://nmap.org ) at 2011-10-25 19:29 PDT
Interesting ports on localhost (127.0.0.1):
PORT      STATE  SERVICE
1417/tcp  open  timbuku-srv1
```

Nmap done: 1 IP address (1 host up) scanned in 0.135 seconds

Now, here's an Nmap version scan getting it right:

```
[root$] nmap -sV target.com -p 1417
```

```
Starting Nmap 4.68 ( http://nmap.org ) at 2011-10-25 19:25 PDT
Interesting ports on localhost (127.0.0.1):
PORT      STATE  SERVICE  VERSION
1417/tcp  open  ssh      OpenSSH 3.7
```

Service detection performed. Please report any incorrect results at <http://nmap.org/submit/>.

Nmap done: 1 IP address (1 host up) scanned in 0.981 seconds



Amap Version Scanning

Popularity:	9
Simplicity:	8
Impact:	3
Risk Rating:	7

Amap (thc.org/thc-amap/) is a dedicated service fingerprinting tool, the first of its kind, predating the Nmap version scanning functionality discussed above by years. At the time of this writing, largely due to its vast preexisting user and developer base, Nmap has since gone on to become the premier version scanning tool. But when fingerprinting services, sometimes getting a second opinion is helpful. Amap utilizes its own network service pattern-matching techniques to fingerprint network services, and although Nmap's functionality is typically more accurate and up-to-date, occasionally Amap catches something Nmap has difficulty with.

VULNERABILITY SCANNERS

When stealth isn't required, whether because the attacker knows the target doesn't have effective monitoring capabilities or she is simply moving quickly enough not to be concerned about detection, employing the battering-ram approach of directing an automated vulnerability scanner against a target or entire network can be an effective and time-efficient means of gathering vulnerability information.

Typically, automated vulnerability scanners contain and regularly update vast databases of known vulnerability signatures for essentially anything listening on a network port, including operating systems, services, and web applications. They can even detect vulnerabilities in client-side software given sufficient credentials, an approach that may be useful in later stages of the attack when the attacker may be interested in expanding her foothold further by compromising additional privileged user accounts.

Numerous vulnerability scanning tools are available commercially at the time of this writing, from companies including McAfee, Qualys, Rapid7, nCircle, and Tenable. On the open source front, the Open Vulnerability Assessment System (OpenVAS, openvas.org) is an alternative for those looking for free tools. We describe one of the more popular tools next to demonstrate the capability of modern scanners to perform enhanced enumeration.



Nessus Scanning

<i>Popularity:</i>	9
<i>Simplicity:</i>	9
<i>Impact:</i>	6
<i>Risk Rating:</i>	8

Nessus, by Tenable Network Security (nessus.org/products/nessus), has long been the gold standard of vulnerability scanners. Its easy-to-use graphical interface, frequently updated database of vulnerabilities, support for all major platforms (the Nessus client component has even been ported to iPhone and Android!), and optimized performance make it well suited for exhaustively scanning a target or network of targets in short order. Users can also develop custom plug-ins using the interpreted Nessus Attack Scripting Language (NASL) to extend its capabilities to meet most any imaginable scanning need. Figure 3-1 shows the Nessus web console.

NOTE

Be sure you are in compliance with Nessus's licensing model, particularly if you plan to use recent versions of it in a corporate setting. Nessus was free and open source until version 3 when it changed to a proprietary closed-source model. Because of this, some users have preferred to stay with Nessus 2 or the open source, community-driven alternative that forked out of Nessus 2, OpenVAS (openvas.org). But recent improvements to Nessus's scanning engine and plug-ins make the newer releases compelling and most likely worthy of the investment. As of this writing, home users could use the Nessus 4 HomeFeed for free, but corporate users must purchase the ProfessionalFeed.

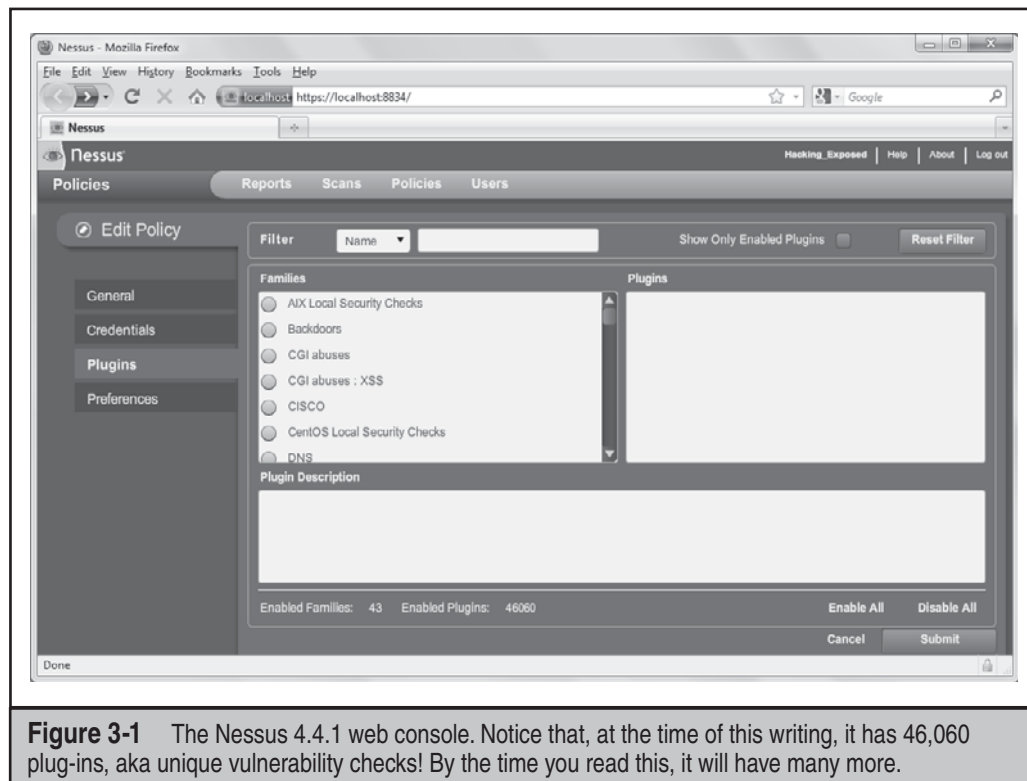


Figure 3-1 The Nessus 4.4.1 web console. Notice that, at the time of this writing, it has 46,060 plug-ins, aka unique vulnerability checks! By the time you read this, it will have many more.



Nessus Scanning Countermeasures

To prevent your system's vulnerabilities from being enumerated by tools like Nessus, you should, of course, implement effective patch and configuration management processes to try to prevent such vulnerabilities from being introduced in the first place. But also regularly scan your own systems with such tools, so you can detect and remediate the ones that get through, hopefully before an attacker has the opportunity.

In addition, due to the sheer popularity of automated vulnerability scanners, Intrusion Detection and Prevention System (IDS/IPS) vendors have tuned their detection signatures to alert on the behavior of tools like Nessus. In the case of IPS, products can block or simply slow scans down to a crawl, frustrating the attacker, which may cause him to move on to the next, softer target if he is simply an opportunistic individual.



Nmap NSE Scripting

<i>Popularity:</i>	7
<i>Simplicity:</i>	6
<i>Impact:</i>	5
<i>Risk Rating:</i>	6

As if Nmap wasn't powerful enough, it also has the ability to conduct all of the enumeration activities covered in this chapter and so much more via the Nmap Scripting Engine (NSE).

Nmap's NSE is an interface that allows users to extend Nmap's capabilities via their own custom scripts written in the Lua interpreted programming language to send, receive, and report on arbitrary data. This feature clearly creates some overlap between Nmap and tools like Nessus. But as stated on nmap.org, this functionality was not introduced so Nmap could compete head-to-head with Nessus (why reinvent the wheel after all?) but rather so it could be utilized to check for specific issues, typically when a scalpel is preferred to a battering ram.

Nmap comes bundled with a library of useful NSE scripts (invoked by adding either `--script` to run a specific script or `-sC` to run a set of default scripts) capable of performing activities such as network discovery, version detection, backdoor detection, and even exploitation of vulnerabilities. The following demonstrates an SMB vulnerability checker Nmap NSE script, which comes bundled with current versions of Nmap (note this script even has an option to enable unsafe, i.e., potentially disruptive, tests):

```
[root$] nmap -Pn --script smb-check-vulns --script-args=unsafe=1 192.168.1.3
```

```
Starting Nmap 5.21 ( http://nmap.org ) at 2011-11-26 18:57 PST
NSE: Script Scanning completed.
Nmap scan report for test-jg7wfg6i5r.ftdrhcpuser.net (192.168.1.3)
Host is up (1.0s latency).
Not shown: 994 closed ports
PORT      STATE      SERVICE
135/tcp   open       msrpc
139/tcp   open       netbios-ssn
445/tcp   open       microsoft-ds
514/tcp   filtered   shell
1025/tcp  open       NFS-or-IIS
5000/tcp  open       upnp

Host script results:
| smb-check-vulns:
|   MS08-067: VULNERABLE
|_  SMBv2 DoS (CVE-2009-3103): VULNERABLE

Nmap done: 1 IP address (1 host up) scanned in 716.68 seconds
```

BASIC BANNER GRABBING

The most fundamental of enumeration techniques is *banner grabbing*, which was mentioned briefly in Chapter 2. Banner grabbing can be simply defined as connecting to remote services and observing the output, and it can be surprisingly informative to remote attackers. At the very least, they may identify the make and model of the running service, which in many cases is enough to set the vulnerability research process in motion.

As also noted in Chapter 2, many port-scanning tools can perform banner grabbing in parallel with their main function of identifying open ports (the harbinger of an exploitable remote service). This section briefly catalogs the most common *manual* techniques for banner grabbing, of which no self-respecting hacker should be ignorant (no matter how automated port scanners become).



The Basics of Banner Grabbing: telnet and netcat

Popularity:	5
Simplicity:	9
Impact:	1
Risk Rating:	5

The tried-and-true manual mechanism for enumerating banners and application info has traditionally been based on telnet (a remote communications tool built into most operating systems). Using telnet to grab banners is as easy as opening a telnet connection to a known port on the target server, pressing ENTER a few times, if necessary, and seeing what comes back:

```
C:\>telnet www.example.com 80
```

```
HTTP/1.1 400 Bad Request
Server: Microsoft-IIS/5.0
Date: Tue, 15 Jul 2008 21:33:04 GMT
Content-Type: text/html
Content-Length: 87
```

```
<html><head><title>Error</title>
</head><body>The parameter is incorrect. </body>
</html>
```

This is a generic technique that works with many common applications that respond on a standard port, such as HTTP port 80, SMTP port 25, or FTP port 21.

For a slightly more surgical probing tool, rely on netcat, the “TCP/IP Swiss Army knife.” Netcat was written by Hobbitt and ported to the Windows NT Family by Weld Pond while he was with the L0pht security research group. As you will see throughout this book, netcat belongs in the permanent System Administrators Hall of Fame for its elegant flexibility. When employed by the enemy, it is simply devastating. Here, we

examine one of its more simplistic uses, connecting to a remote TCP/IP port and enumerating the service banner:

```
C:\>nc -v www.example.com 80
www.example.com [10.219.100.1] 80 (http) open
```

A bit of input here usually generates some sort of a response. In this case, pressing ENTER causes the following:

```
HTTP/1.1 400 Bad Request
Server: Microsoft-IIS/5.0
Date: Tue, 15 Jul 2008 00:55:22 GMT
Content-Type: text/html
Content-Length: 87

<html><head><title>Error</title>
</head><body>The parameter is incorrect. </body>
</html>
```

One tip from the netcat readme file discusses how to redirect the contents of a file into netcat to nudge remote systems for even more information. For example, create a text file called `nudge.txt` containing the single line `GET / HTTP/1.0`, followed by two carriage returns, and then the following:

```
[root$]nc -nvv -o banners.txt 10.219.100.1 80 < nudge.txt
(unknown) [10.219.100.1] 80 (http) open

HTTP/1.1 200 OK
Server: Microsoft-IIS/5.0
Date: Wed, 16 Jul 2008 01:00:32 GMT
X-Powered-By: ASP.NET
Connection: Keep-Alive
Content-Length: 8601
Content-Type: text/html
Set-Cookie: ASPSESSIONIDCCRRABCR=BEFOAIJDCHMLJENPIPJGJACM; path=/
Cache-control: private

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
http://www.w3.org/TR/xhtml1/DTD/xhtml
11-transitional.dtd">
<HTML>
<HEAD>
  <META NAME="keywords" CONTENT=" Example, Technology ">
  <META NAME="description" CONTENT="Welcome to Example's Web site. ">
<TITLE>Example Corporate Home Page</TITLE>
</HEAD>
</HTML>
```

TIP

The `netcat -n` argument is recommended when specifying numeric IP addresses as a target.

Know any good exploits for Microsoft IIS 5.0? You get the point. Depending on the service being probed, the nudge file can contain various possibilities, such as `HEAD / HTTP/1.0 <cr><cr>`, `QUIT <cr>`, `HELP <cr>`, `ECHO <cr>`, or even just a couple carriage returns (`<cr>`).

This information can significantly focus an intruder's effort to compromise a system. Now that the vendor and version of the server software are known, attackers can concentrate on platform-specific techniques and known exploit routines until they get one right. Time is shifting in their favor and against the administrator of this machine. You'll hear more about netcat throughout this book.



Banner-Grabbing Countermeasures

As we've already noted, the best defense against banner grabbing is to shut down unnecessary services. Alternatively, restrict access to services using network access control. Perhaps the widest avenue of entry into any environment is running vulnerable software services, so this access should be restricted to combat more than just banner grabbing.

Next, for those services that are business critical and can't simply be turned off, you need to research the correct way to disable the presentation of the vendor and version in banners. Audit yourself regularly with automated tools and manual spot checks (e.g., with netcat) to make sure you aren't giving away inappropriate information to attackers.

ENUMERATING COMMON NETWORK SERVICES

Let's use some of these basic enumeration techniques, and much more, to enumerate services commonly turned up by real-world port scans.



FTP Enumeration, TCP 21

<i>Popularity:</i>	1
<i>Simplicity:</i>	10
<i>Impact:</i>	1
<i>Risk Rating:</i>	4

Although File Transfer Protocol (FTP) is becoming less common on the Internet, connecting to and examining the content of FTP repositories remains one of the simplest and potentially lucrative enumeration techniques. We've seen many public web servers that used FTP for uploading web content, providing an easy vector for uploading

malicious executables (see Chapter 10 on web hacking for more details). Typically, the availability of easily accessible file-sharing services quickly becomes widespread knowledge, and public FTP sites end up hosting sensitive and potentially embarrassing content. Even worse, many such sites are configured for anonymous access.

Connecting to FTP is simple, using the client that is typically built into most modern operating systems. The next example shows the Windows command-line FTP client. Note that we use “anonymous” and a spurious e-mail address (not shown in the following output) to authenticate to this anonymous service:

```
C:\>ftp ftp.example.com
Connected to ftp.example.com.
220 (vsFTPd 2.0.1)
User (ftp.example.com:(none)): anonymous
331 Please specify the password.
Password:
230 Login successful.
ftp> ls
200 PORT command successful. Consider using PASV.
150 Here comes the directory listing.
GO
DROP
hos2
hml
LINK
lib
lost+found
pub
226 Directory send OK.
ftp: 52 bytes received in 0.00Seconds 52000.00Kbytes/sec.
ftp>
```

Of course, graphical FTP clients are also available. Most modern web browsers implement FTP and permit browsing of sites via the familiar file-and-folder metaphor. An excellent open source graphical FTP client is FileZilla from filezilla-project.org/. For a list of anonymous FTP sites, see ftp-sites.org. Although this site hasn’t been recently updated, it does contain many sites that are still available.

And, of course, the banner enumerated by FTP can indicate the presence of FTP server software with severe vulnerabilities. Washington University’s FTP server (wu-ftp), for example, was once very popular with attackers due to its history of remotely exploitable buffer overflows that permit complete compromise of the system.



FTP Enumeration Countermeasures

FTP is one of those “oldie-but-not-so-goodie-anymore” services that should just be turned off. Always use Secure FTP (SFTP, which utilizes SSH encryption) or FTP Secure

(FTPS, which utilizes SSL) protected by strong passwords or certificate-based authentication. Be especially skeptical of anonymous FTP, and don't allow unrestricted uploading of files under any circumstances. And public content is often better served via HTTP rather than file-sharing protocols altogether.



Enumerating Telnet, TCP 23

Popularity:	4
Simplicity:	9
Impact:	3
Risk Rating:	5

Telnet was one of the most crucial services in use for many years. In the early days of the Internet, telnet was so valuable because it provided one of the most essential services: remote access. Telnet's major downfall is that it transmits data in *cleartext*. This means that anyone with a sniffer can potentially view the entire conversation between the client and server, including the username and password used to log in. With security becoming more of a necessity, this service was later replaced by a more secure, encrypted means of remote administration called *secure shell*, or *SSH*. Even though telnet's insecurities are widely known, this service is still commonly available.

System Enumeration via Telnet Banners From an attacker's standpoint, telnet can be an easy way to obtain host information because telnet usually displays a system banner prior to login. This banner often contains the host's operating system and version. With networking equipment such as routers and switches, you may not receive such an explicitly detailed banner. Many times the system displays a unique prompt from which you can easily deduce what type of device it is through prior knowledge or a simple Google search. For instance, with Cisco equipment, you'll receive one of two prompts:

```
User Access Verification.  
Password:  
Or
```

```
User Access Verification.  
Username:
```

If you receive either banner, you can pretty safely assume that the host you're connecting to is a Cisco device. The difference between the two prompts is that the Username prompt on Cisco telnet servers usually indicates that the device is using TACACS+ or some sort of authentication, authorization, and accounting (AAA) for authentication, which means some set of lockout mechanisms are most likely in place. This information can aid an attacker in choosing an attack plan when brute forcing. In the case that only a password is requested, the attacker can very likely launch a brute-

force attack without being locked out and, in many cases, go unnoticed by the owner of the device.

Account Enumeration via Telnet As you're learning in this chapter, services, daemons, and all other types of client-facing applications can provide valuable information if you just know how to ask for it and what response to look for. One perfect example of this is account enumeration, which is the process of attempting to log in with a particular username and observing the error messages returned by the server. One instance of account enumeration via telnet was demonstrated by Shalom Carmel at Black Hat Europe during his presentation "AS/400 for Pentesters." Shalom showed that the AS/400 allows for username enumeration during telnet authentication (and POP3). For instance, if an attacker attempts to log in with a valid username but an invalid password, the system responds with "CPF1107 – Password not correct for user profile." If an attacker attempts to log in with an invalid username, the system responds "CPF 1120 – User X does not exist." By harvesting the responses from the server for particular usernames, the attacker can begin to build a list of valid accounts for brute forcing. Shalom also provided a list of other common but useful AS/400 error messages provided during authentication, as shown in Table 3-1.

Telnet Enumeration Countermeasures

Generally speaking, the insecure nature of telnet should be cause enough to discontinue its use and seek alternate means of remote management. Secure shell (SSH) is a widely deployed alternative that should be used as a replacement in all possible cases. In situations where telnet must be used, mitigating controls to restrict access to the service on a host/segment basis should be deployed. Banner information can be modified in

Error	Message
CPF1107	Password not correct for user profile
CPF1109	Not authorized to subsystem
CPF1110	Not authorized to work station
CPF1116	Next not valid sign-on attempt varies off device
CPF1118	No password associated with user X
CPF1120	User X does not exist
CPF1133	Value X is not a valid name
CPF1392	Next not valid sign-on disables user profile
CPF1394	User profile X cannot sign in

Table 3-1 Common Error Messages

most cases, so be sure to consult your vendor for more information. In regards to the specific AS/400 telnet enumeration issue, these error messages can be modified to be generalized using the CHMSGD command, and it is recommended you require users to reconnect between failed login attempts.



Enumerating SMTP, TCP 25

<i>Popularity:</i>	5
<i>Simplicity:</i>	9
<i>Impact:</i>	1
<i>Risk Rating:</i>	5

One of the most classic enumeration techniques takes advantage of the lingua franca of Internet mail delivery, the Simple Mail Transfer Protocol (SMTP), which typically runs on TCP port 25. SMTP provides two built-in commands that allow for the enumeration of users: `VRFY`, which confirms names of valid users, and `EXPN`, which reveals the actual delivery addresses of aliases and mailing lists. Although most companies give out e-mail addresses quite freely these days, allowing this activity on your mail server raises the possibility of forged e-mail and, more importantly, can provide intruders with the names of local user accounts on the server. We use telnet in the next example to illustrate SMTP enumeration, but you can use netcat as well:

```
[root$] telnet 10.219.100.1 25
Trying 10.219.100.1...
Connected to 10.219.100.1.
Escape character is '^]'.
220 mail.example.com ESMTP Sendmail Tue, 15 Jul 2008 11:41:57
vrfy root
250 root <root@mail.example.com>
expn test
250 test <test@mail.example.com>
expn non-existent
550 5.1.1 non-existent... User unknown
quit
221 mail.example.com closing connection
```

A tool called `vrfy.pl` can speed up this process. An attacker can use `vrfy.pl` to specify the target SMTP server and a list of usernames to test. `vrfy.pl` then runs through the username file and reports back on which users the server has identified as valid.



SMTP Enumeration Countermeasures

This is another one of those oldie-but-goodie services that should just be turned off. Versions of the popular SMTP server software sendmail (sendmail.org) greater than 8

offer syntax that can be embedded in the mail.cf file to disable these commands or require authentication. Microsoft's Exchange Server prevents nonprivileged users from using EXPN and VRFY, by default, in more recent versions. Other SMTP server implementations should offer similar functionality. If they don't, consider switching vendors!



DNS, TCP/UDP 53

<i>Popularity:</i>	5
<i>Simplicity:</i>	9
<i>Impact:</i>	2
<i>Risk Rating:</i>	5

As you saw in Chapter 1, one of the primary sources of footprinting information is the Domain Name System (DNS), the Internet standard protocol for matching host IP addresses with human-friendly names such as “foundstone.com.” DNS normally operates on UDP port 53 but may also run on TCP port 53 for extended features such as zone transfers.

DNS Enumeration with Zone Transfers One of the oldest enumeration techniques is the DNS zone transfer, which can be implemented against misconfigured DNS servers via TCP port 53. Zone transfers dump the entire contents of a given domain's zone files, enumerating information such as hostname-to-IP address mappings as well as Host Information Record (HINFO) data (see Chapter 1).

If the target server is running Microsoft DNS services to support Active Directory (AD), there's a good chance an attacker can gather even more information. Because the AD namespace is based on DNS, Microsoft's DNS server implementation advertises domain services such as AD and Kerberos using the DNS SRV record (RFC 2052), which allows servers to be located by service type (for example, LDAP, FTP, or WWW) and protocol (for example, TCP). Therefore, a simple zone transfer (`nslookup`, `ls -d <domainname>`) can enumerate a lot of interesting network information, as shown in the following sample zone transfer run against the domain “example2.org” (edited for brevity and line-wrapped for legibility):

```
C:\>nslookup
Default Server: ns1.example.com
Address: 10.219.100.1
> server 192.168.234.110

Default Server: corp-dc.example2.org
Address: 192.168.234.110

> ls -d example2.org
[[192.168.234.110]]
example2.org. SOA corp-dc.example2.org admin.
```

```

example2.org.          A      192.168.234.110
example2.org.          NS      corp-dc.example2.org
. . .
_gc._tcp               SRV priority=0, weight=100, port=3268, corp-dc.example2.org
_kerberos._tcp         SRV priority=0, weight=100, port=88, corp-dc.example2.org
_kpasswd._tcp          SRV priority=0, weight=100, port=464, corp-dc.example2.org
_ldap._tcp             SRV priority=0, weight=100, port=389, corp-dc.example2.org

```

Per RFC 2052, the format for SRV records is as follows:

```
Service.Proto.Name TTL Class SRV Priority Weight Port Target
```

Some very simple observations an attacker could take from this file would be the location of the domain's Global Catalog service (`_gc._tcp`), domain controllers using Kerberos authentication (`_kerberos._tcp`), LDAP servers (`_ldap._tcp`), and their associated port numbers. (Only TCP incarnations are shown here.)

Alternatively, from within Linux (or other UNIX variants), we can use the `dig` command to produce similar results:

```

~ $ dig @192.168.234.110 example2.org axfr

; <<>> DiG 9.3.2 <<>> @192.168.234.110 example2.org axfr
; (1 server found)
;; global options: printcmd
example2.org.      86400 IN      SOA      corp-dc.example2.org admin.
example2.org.      86400 IN      A        192.168.234.110
example2.org.      86400 IN      NS       corp-dc.example2.org
. . .
_gc._tcp           86400 IN      SRV      0 100 3268 corp-dc.example2.org
_kerberos._tcp     86400 IN      SRV      0 100 88 corp-dc.example2.org
_kpasswd._tcp      86400 IN      SRV      0 100 464 corp-dc.example2.org
_ldap._tcp         86400 IN      SRV      0 100 389 corp-dc.example2.org
;; Query time: 489 msec
;; SERVER: 192.168.234.110#53(192.168.234.110)
;; WHEN: Wed Jul 16 15:10:27 2008
;; XFR size: 45 records (messages 1)

```

BIND Enumeration The Berkeley Internet Name Domain (BIND) server is a popular DNS server for UNIX variants. In addition to being susceptible to DNS zone transfers, BIND comes with a record within the “CHOAS” class, `version.bind`, which contains the version of the BIND installation loaded on the target server. To request this record, the attacker can use the `dig` command:

```

~ $ dig @10.219.100.1 version.bind txt chaos

; <<>> DiG 9.3.2 <<>> @10.219.100.1 version.bind txt chaos

```



```
; (1 server found)
;; global options: printcmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 1648
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;version.bind.                CH      TXT

;; ANSWER SECTION:
version.bind.                0      CH      TXT      "9.2.4"

;; Query time: 399 msec
;; SERVER: 10.219.100.1#53(10.219.100.1)
;; WHEN: Wed Jul 16 19:00:04 2008
;; MSG SIZE rcvd: 48
```

DNS Cache Snooping DNS servers maintain a cache for a variety of reasons, one of which is to resolve frequently used hostnames quickly. For requests to resolve hostnames not within the target DNS server's domain, the DNS server queries its local cache or uses recursion to resolve the request by querying another DNS server. Attackers can abuse this functionality by requesting the DNS server to query only its cache and, by doing so, deduce if the DNS server's clients have or have not visited a particular site. In the case that the DNS server hasn't processed a request for a particular host, the server responds with the "Answer" flag set to 0 (output has been condensed):

```
~ $ dig @10.219.100.1 www.foundstone.com A +norecurse
; <<>> DiG 9.3.2 <<>> @10.219.100.1 www.foundstone.com A +norecurse
; (1 server found)
;; global options: printcmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 4954
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 13, ADDITIONAL: 13

;; QUESTION SECTION:
;www.foundstone.com.         IN      A

;; AUTHORITY SECTION:
com.                        161611  IN      NS      A.GTLD-SERVERS.NET.

;; ADDITIONAL SECTION:
A.GTLD-SERVERS.NET.        111268  IN      A      192.5.6.30

;; Query time: 105 msec
;; SERVER: 10.219.100.1#53(10.219.100.1)
```

```
;; WHEN: Wed Jul 16 19:48:27 2008
;; MSG SIZE rcvd: 480
```

Once the DNS server has processed a request for the particular hostname, the “Answer” flag is then set to 1:

```
~ $ dig @10.219.100.1 www.foundstone.com A +norecurse

; <<>> DiG 9.3.2 <<>> @10.219.100.1www.foundstone.com A +norecurse
; (1 server found)
;; global options: printcmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 16761
;; flags: qr ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;www.foundstone.com.          IN      A

;; ANSWER SECTION:
www.foundstone.com.          297     IN      A      216.49.88.17

;; Query time: 103 msec
;; SERVER: 10.219.100.1#53(10.219.100.1)
;; WHEN: Wed Jul 16 19:57:24 2008
;; MSG SIZE rcvd: 52
```

Automated DNS Enumeration Various DNS tools exist that will automate the preceding enumeration techniques and perform a number of different tasks that may give you additional information about a domain and the hosts within it. `dnsenum` (code.google.com/p/dnsenum/), written by Filip Waeytens and tixxDZ, does a variety of different tasks, such as Google scrapping for additional names and subdomains, brute forcing subdomains, performing reverse lookups, listing domain network ranges, and performing WHOIS queries on the ranges identified. The power of `dnsenum` comes from the correlation it performs across each task to gather as much information for a particular domain as possible. The tool can be run on a domain name; it then deduces the DNS servers associated with it. It can also be run against a target server for a particular domain.

Another powerful automated DNS reconnaissance tool is `Fierce.pl` (ha.ckers.org/fierce/), a Perl script written by Robert “RSnake” Hansen that uses a number of techniques to locate IP addresses and hostnames owned by a target, including attempting zone transfers, dictionary list, and brute-force reverse lookup enumeration.

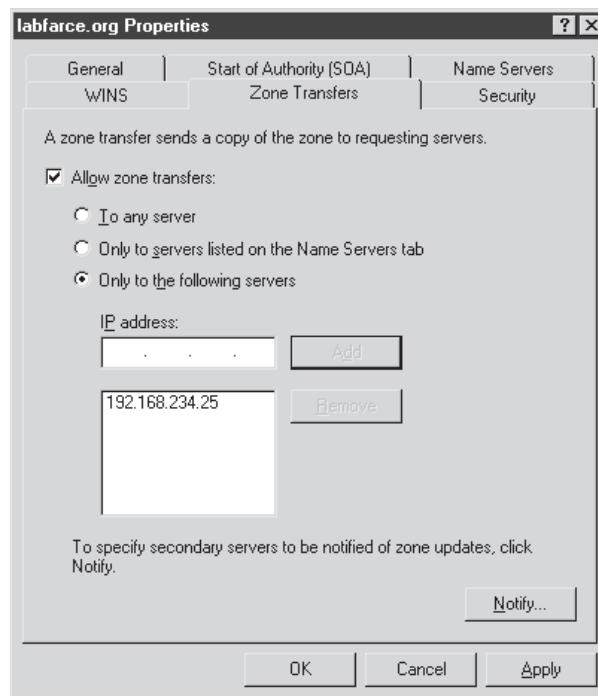
Also, web resources exist that not only speed up and simplify the process but also give the attacker the advantage of not having to send a single packet to the target from the source IP address. Rather, the attacker stays hidden behind the public resource. The

site CentralOps.net hosts a number of free reconnaissance tools, including WHOIS enumeration, zone transferring, and even service scanning.

— DNS Enumeration Countermeasures

As always, if DNS is not required, the best countermeasure is simply to disable the service. However, you will very likely need an Internet-facing DNS server on your perimeter to maintain business operations. In addition to thwarting the specific techniques just described, maintaining two DNS servers is important: one for external, Internet-facing queries and one for internal queries. With this countermeasure, if a vulnerability or misconfiguration is identified within your public-facing DNS server, internal addressing and critical targets are not exposed.

Blocking DNS Zone Transfers The easy solution for this problem is to restrict zone transfers to authorized machines only (usually, these are backup DNS servers). The Windows DNS implementation allows for easy restriction of zone transfer, as shown in the following illustration. This screen is available when the Properties option for a forward lookup zone (in this case, labfarce.org) is selected from within the “Computer Management” Microsoft Management Console (MMC) snap-in, under \Services and Applications\DNS\[server_name]\Forward Lookup Zones\[zone_name] | Properties.



You could disallow zone transfers entirely by simply unchecking the Allow Zone Transfers box, but it is probably more realistic to assume that backup DNS servers will need to be kept up to date, so we have shown a less restrictive option here.

NOTE

Past versions of Windows (up to and including Windows 2000) came configured, by default, to allow zone transfers to any server. However, thanks in part to the depiction of this issue in past editions of *Hacking Exposed*, Microsoft released its later server versions with a default DNS server setting that blocks zone transfers to unauthorized systems. Hats off to Redmond!

Blocking BIND version.bind Requests An excellent BIND hardening guide by Rob Thomas is available at cymru.com/Documents/secure-bind-template.html. This guide includes a number of different methods to secure BIND, including how to change or disable queries for version.bind.

Disabling DNS Cache-Snooping Luis Grangeia has written a paper (rootsecure.net/content/downloads/pdf/dns_cache_snooping.pdf) that further describes DNS cache snooping and provides methods to protect against it.



Enumerating TFTP, TCP/UDP 69

Popularity:	1
Simplicity:	3
Impact:	7
Risk Rating:	3

Trivial File Transfer Protocol (TFTP) is a UDP-based protocol for unauthenticated “quick and dirty” file transfers commonly run on UDP port 69. The premise of TFTP is that in order to pull a file from a server, you have to know the file name. This can be a double-edged sword for an attacker because the results are not always guaranteed. For instance, if the file has been renamed by even a single character, the attacker’s request will fail.

Copying Files via a Linux TFTP Server Although it barely qualifies as an enumeration trick due to the severity of the information gathered, the granddaddy of all UNIX/Linux enumeration tricks is getting the `/etc/passwd` file, which we’ll discuss at length in Chapter 5. However, it’s worth mentioning here that one way to grab the `passwd` file is via TFTP. It’s trivial to grab a poorly secured `/etc/passwd` file via TFTP, as shown next:

```
[root$] tftp 192.168.202.34
tftp> connect 192.168.202.34
tftp> get /etc/passwd /tmp/passwd.cracklater
tftp> quit
```

Besides the fact that our attackers now have the `passwd` file to view all valid user accounts on the server, if this were an older system, they could potentially gain access to the encrypted password hashes for each user. On newer systems, they might find it worthwhile to attempt to transfer the `/etc/shadow` file as well.

Accessing Router/Switch Configurations via TFTP Network devices such as routers, switches, and VPN concentrators commonly provide the functionality to configure the device as a TFTP server. In some cases, attackers can leverage this functionality to their advantage in order to obtain the device's configuration file. Files an attacker may look for on network devices include

```
running-config
startup-config
.config
config
run
```



TFTP Enumeration Countermeasures

TFTP is an inherently insecure protocol—the protocol runs in cleartext on the wire, it offers no authentication mechanism, and it can leave misconfigured file-system ACLs wide open to abuse. For these reasons, don't run TFTP—and if you do, wrap it to restrict access (using a tool such as TCP Wrappers), limit access to the `/tftpboot` directory, and make sure it's blocked at the border firewall.



Finger, TCP/UDP 79

<i>Popularity:</i>	7
<i>Simplicity:</i>	10
<i>Impact:</i>	1
<i>Risk Rating:</i>	6

Perhaps the oldest trick in the book when it comes to enumerating users is the UNIX/Linux `finger` utility. `Finger` was a convenient way of giving out user information automatically back in the days of a much smaller and friendlier Internet. We discuss it here primarily to describe the attack signature because many scripted attack tools still try it and many unwitting system admins leave `finger` running with minimal security configurations. Again, the following assumes that a valid host running the `finger` service (port 79) has been identified in previous scans:

```
[root$] finger -l @target.example.com
[target.example.com]
Login: root                               Name: root
Directory: /root                          Shell: /bin/bash
```

```

On since Sun Mar 28 11:01 (PST) on tty1 11 minutes idle
(messages off)
On since Sun Mar 28 11:01 (PST) on ttty0 from :0.0
  3 minutes 6 seconds idle
No mail.
plan:
John Smith
Security Guru
Telnet password is my birthdate.

```

finger 0@hostname also turns up good info:

```

[root$]finger 0@192.168.202.34
[192.168.202.34]
      Line      User      Host(s)      Idle Location
*   2 vty 0           idle          0 192.168.202.14
      Se0              Sync PPP    00:00:02

```

As you can see, most of the info displayed by finger is fairly innocuous. (It is derived from the appropriate /etc/passwd fields if they exist.) Perhaps the most dangerous information contained in the finger output is the names of logged-on users and idle times, giving attackers an idea of who's watching (root?) and how attentive they are. Some of the additional information could be used in a "social engineering" attack (hacker slang for trying to con access from people using "social" skills). As noted in this example, users who place a .plan or .project file in their home directories can deal potential wildcards of information to simple probes. (The contents of such files are displayed in the output from finger probes, as shown earlier.)



Finger Countermeasures

Detecting and plugging this information leak is easy—don't run finger (comment it out in inetd.conf and killall -HUP inetd) and block port 79 at the firewall. If you must (and we mean *must*) give access to finger, use TCP Wrappers (see Chapter 5) to restrict and log host access, or use a modified finger daemon that presents limited information.



Enumerating HTTP, TCP 80

Popularity:	5
Simplicity:	9
Impact:	1
Risk Rating:	5

Enumerating the make and model of a web server is one of the easiest and most time-honored techniques of the hacking community. Whenever a new web server exploit is released into the wild (for example, the old ida/idq buffer overflow that served as the

basis for the Code Red and Nimda worms), the underground turns to simple, automated enumeration tools to check entire swaths of the Internet for potentially vulnerable software. Don't think you won't get caught.

We demonstrated elementary HTTP banner grabbing at the beginning of this chapter in the section titled "The Basics of Banner Grabbing: telnet and netcat." In that section, we showed you how to connect to a web server on the standard HTTP port (TCP 80) using netcat and how to hit a few carriage returns to extract the banner. Usually the HTTP HEAD method is a clean way to elicit banner info. You can type this command right into netcat once you've connected to the target server, as shown here (commands to be entered are listed in bold; you'll need to hit two or more carriage returns after the line containing the HEAD command):

```
C:\>nc -v www.example.com 80
www.example.com [10.219.100.1] 80 (http) open
HEAD / HTTP/1.1

HTTP/1.1 200 OK
Server: Microsoft-IIS/5.0
Date: Thu, 17 Jul 2008 14:14:50 GMT
X-Powered-By: ASP.NET
Content-Length: 8601
Content-Type: text/html
Set-Cookie: ASPSESSIONIDCCRRABCR=MEJICIJDLAMKPGOIJAFBJOGD; path=/
Cache-control: private
```

We demonstrated the HTTP HEAD request in the previous example, which is uncommon nowadays, with the notable exception of worms. Therefore, some intrusion detection systems might trigger from a HEAD request.

Also, if you encounter a website that uses SSL, don't fret, because netcat can't negotiate SSL connections. Simply redirect it through one of the many available SSL proxy tools, such as sslproxy, or just use openssl to perform the task:

```
~ $ openssl s_client -quiet -connect www.example.com:443

HEAD / HTTP/1.1
host: www.example.com

HTTP/1.1 200 OK
Server: Microsoft-IIS/5.0
Date: Thu, 17 Jul 2008 14:22:13 GMT
X-Powered-By: ASP.NET
Content-Length: 8601
Content-Type: text/html
Set-Cookie: ASPSESSIONIDAADQDAAQ=BEMJCIICCJBGGKCLLOIBBOHA; path=/
Cache-control: private
```

By default, `openssl` is extremely verbose, so specify the `-quiet` switch to limit its output. You may notice that we've also specified `host: www.example.com` after our `HEAD / HTTP/1.1` nudge. We did this because servers have the ability to host multiple websites, so in some cases you may have to set the HTTP host header to the hostname of the web page you're visiting to elicit a 200 OK (or "request succeeded" code) from the web server. For this particular example, the web server will provide its versioning information for just about any HTTP request, but when you start getting into more advanced techniques, the HTTP host header may save some heartache.

We should point out here that much juicy information can also be found in the content of web pages. One of our favorite automated tools for crawling entire sites and reporting on matches to a set of known vulnerabilities is Grendel-Scan by David Byrne (grendel-scan.com/download.htm). Figure 3-2 shows Grendel-Scan's Information Leakage section, containing features such as the ability to suck down all of the comments in a website so an attacker may search these comments for juicy information such as the phrase "password" or the ability to parse a website's robots.txt file and pay particular attention to its entries—potentially interesting web content identified for one reason or another by its author as not appropriate for search engine indexing.

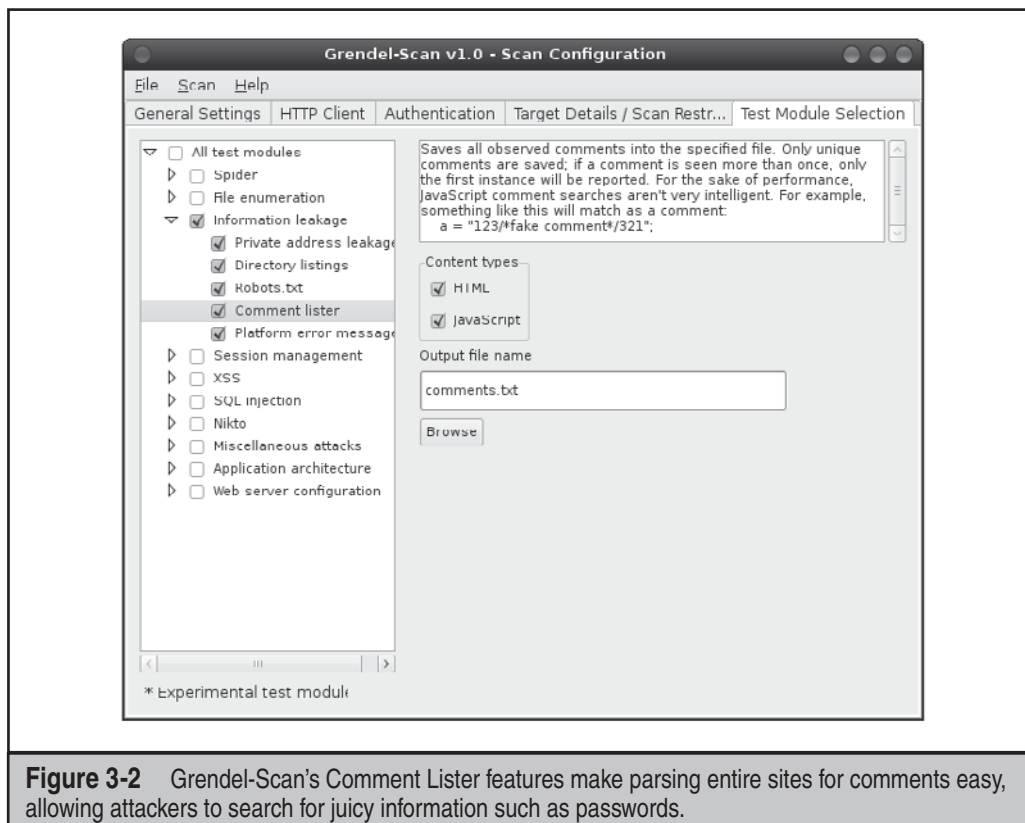


Figure 3-2 Grendel-Scan's Comment Lister features make parsing entire sites for comments easy, allowing attackers to search for juicy information such as passwords.

Crawling HTML for juicy information edges into the territory of web hacking, which we cover in Chapter 10 of this book.

TIP

For an expanded and more in-depth examination of web hacking methodologies, tools, and techniques, check out *Hacking Exposed Web Applications, Third Edition* (McGraw-Hill Professional, 2010; webhackingexposed.com).



HTTP Enumeration Countermeasures

The best way to deter this sort of activity is to change the banner on your web servers. Steps to do this vary depending on the web server vendor, but we'll illustrate using one of the most common examples—Microsoft's Internet Information Services (IIS). In the past, IIS was frequently targeted, due primarily to the easy availability of canned exploits for debilitating vulnerabilities such as Code Red and Nimda. Changing the IIS banner can go a long way toward dropping you off the radar screen of some really nasty miscreants.

IIS 7 administrators can create a custom .Net module to accomplish this objective, using the example code provided here (some manual line breaks have been added due to page size constraints):

```
using System;
using System.Text;
using System.Web;
namespace HackingExposed.ServerModules
{
    public class CustomServerHeaderModule : IHttpModule
    {
        public void Init(HttpApplication context)
        {
            context.PreSendRequestHeaders += OnPreSendRequestHeaders;
        }
        public void Dispose()
        { }
        void OnPreSendRequestHeaders(object sender, EventArgs e)
        {
            HttpContext.Current.Response.Headers.Set("Server",
                "A Hacking Exposed Reader's Webserver");
        }
    }
}
```

Unfortunately, directly changing the IIS banner in prior IIS versions involves hex-editing the DLL that contains the IIS banner, %systemroot%\system32\inetrv\w3svc.dll. This can be a delicate maneuver, made more difficult on Windows 2000 and later by the fact that this DLL is protected by Windows System File Protection (SFP) and is automatically replaced by a clean copy unless SFP is disabled.

Another way to change the IIS banner on older versions of IIS is by installing an ISAPI filter designed to set the banner using the SetHeader function call. Microsoft has posted a Knowledge Base (KB) article detailing how this can be done, with sample source code, at support.microsoft.com/kb/294735/en-us. Alternatively, you can download and deploy Microsoft's URLScan, part of the IIS Lockdown Tool (see microsoft.com/technet/security/tools/locktool.msp for the IIS Lockdown Tool, applicable to IIS versions prior to 6.0, and microsoft.com/technet/security/tools/urlscan.msp for URLScan, which is applicable to IIS versions up to 6.0). URLScan is an ISAPI filter that can be programmed to block many popular IIS attacks before they reach the web server, and it also allows you to configure a custom banner to fool unwary attackers and automated worms. Deployment and usage of URLScan is fully discussed in *Hacking Exposed Web Applications, Third Edition* (McGraw-Hill Professional, 2010).

NOTE

IIS Lockdown cannot be installed on Windows Server 2003/IIS 6.0 or newer because all the default configuration settings in IIS6.0 (and later) meet or exceed the security configuration settings made by the IIS Lockdown Tool. However, you can install and run URLScan on IIS 6.0 because it provides flexible configuration for advanced administrators above and beyond the default IIS 6.0 security settings. See technet.microsoft.com/en-us/security/cc242650.aspx#EXE.



Enumerating Microsoft RPC Endpoint Mapper (MSRPC), TCP 135

Popularity:	7
Simplicity:	8
Impact:	1
Risk Rating:	5

Certain Microsoft Windows systems run a Remote Procedure Call (RPC) endpoint mapper (or portmapper) service on TCP 135. Querying this service can yield information about applications and services available on the target machine, as well as other information potentially helpful to the attacker. The `epdump` tool from the Windows Resource Kit (RK, or Reskit) queries the MSRPC endpoint mapper and shows services bound to IP addresses and port numbers (albeit in a very crude form). Here's an example of how it works against a target system running TCP 135 (edited for brevity):

```
C:\>epdump mail.example.com
binding is 'ncacn_ip_tcp:mail.example.com'
int 82ad4280-036b-11cf-972c-00aa006887b0 v2.0
    binding 00000000-etc.@ncalrpc:[INETINFO_LPC]
    annot ''
int 82ad4280-036b-11cf-972c-00aa006887b0 v2.0
    binding 00000000-etc.@ncacn_ip_tcp: 105.10.10.126[1051]
    annot ''
int 82ad4280-036b-11cf-972c-00aa006887b0 v2.0
```

```
binding 00000000-etc.@ncacn_ip_tcp:192.168.10.2[1051]
annot ''
no more entries
```

The important thing to note about this output is that we see two numbers that look like IP addresses: 105.10.10.126 and 192.168.10.2. These are IP addresses to which MSRPC applications are bound. More interesting, the second of these is an RFC 1918 address, indicating that this machine likely has two physical interfaces (meaning it is dual-homed) and one of those faces is an internal network. This can raise the interest of curious hackers who seek such bridges between outside and inside networks as key points of attack.

Examining this output further, we note that `ncacn_ip_tcp` corresponds to dynamically allocated TCP ports, further enumerating available services on this system (`ncadg_ip_udp` in the output would correspond to allocated UDP ports). For a detailed and comprehensive explanation of these and other internals of the Windows network services, see Jean-Baptiste Marchand's excellent article at hsc.fr/ressources/articles/win_net_srv.

TIP

Another good MSRPC enumeration tool (and so much more) is Winfingerprint, which can be found at sourceforge.net/projects/winfingerprint.

MSRPC Enumeration with Linux For the Linux side of the house, we have `rpcdump.py` by Javier Koen of CORE security (oss.coresecurity.com/impacket/rpcdump.py). `rpcdump.py` is a little more flexible as it permits queries over different ports/protocols besides TCP 135. Usage is shown here:

```
~ # rpcdump.py
Usage: /usr/bin/rpcdump.py [username[:password]@]<address> [protocol
list...]
Available protocols: ['80/HTTP', '445/SMB', '135/TCP', '139/SMB', '135/UDP']
Username and password are only required for certain transports, eg. SMB.
```



MSRPC Enumeration Countermeasures

The best method for preventing unauthorized MSRPC enumeration is to restrict access to TCP port 135. One area where this becomes problematic is providing mail services via Microsoft Exchange Server to clients on the Internet. In order for Outlook MAPI clients to connect to the Exchange Server, they must first contact the endpoint mapper. Therefore, to provide Outlook/Exchange connectivity to remote users over the Internet, you would have to expose the Exchange Server to the Internet via TCP port 135 (and a variety of others). The most common solution to this problem is to require users to first establish a secure tunnel (that is, using a VPN solution) between their system and the internal network. This way the Exchange Server is not exposed, and data between the client and server is properly encrypted. Of course, the other alternative is to use Microsoft's Outlook Web Access (OWA) to support remote Outlook users. OWA is a web front-end to an

Exchange mailbox, and it works over HTTPS. We recommend using strong authentication if you decide to implement OWA (for example, digital certificates or two-factor authentication mechanisms). In Windows Server 2003/Exchange 2003 (and later), Microsoft implemented RPC over HTTP, which is our favorite option for accessing Exchange over the Internet while preserving the rich look and feel of the full Outlook client (see support.microsoft.com/default.aspx?kbid=833401 and technet.microsoft.com/en-us/library/aa998950.aspx).

If you can't restrict access to MSRPC, you should restrict access to your individual RPC applications. We recommend reading the article titled "Writing a Secure RPC Client or Server" at msdn.microsoft.com/en-us/library/aa379441.aspx for more information on this topic.



NetBIOS Name Service Enumeration, UDP 137

<i>Popularity:</i>	7
<i>Simplicity:</i>	5
<i>Impact:</i>	3
<i>Risk Rating:</i>	5

The NetBIOS Name Service (NBNS) has traditionally served as the distributed naming system for Microsoft Windows-based networks. Beginning with Windows 2000, NBNS is no longer a necessity, having been largely replaced by the Internet-based naming standard, DNS. However, as of this writing, NBNS is still enabled by default in all Windows distributions; therefore, it is generally simple for attackers connected to the local network segment (or via a router that permits the tunneling of NBNS over TCP/IP) to "enumerate the Windows wire," as we sometimes call NBNS enumeration.

NBNS enumeration is so easy because the tools and techniques for peering along the NetBIOS wire are readily available—most are built into the OS itself! In fact, NBNS enumeration techniques usually poll NBNS on all machines across the network and are often so transparent that it hardly appears one is even connecting to a specific service on UDP 137. We discuss the native Windows tools first and then move into some third-party tools. We save the discussion of countermeasures until the very end because fixing all this is rather simple and can be handled in one fell swoop.

Enumerating Windows Workgroups and Domains with net view The `net view` command is a great example of a built-in enumeration tool. It is an extraordinarily simple Windows NT Family command-line utility that lists domains available on the network and then lays bare all machines in a domain. Here's how to enumerate domains on the network using `net view`:

```
C:\>net view /domain
```

```
Domain
```

```
-----  
CORLEONE
```

```
BARZINI_DOMAIN
TATAGGLIA_DOMAIN
BRAZZI
```

The command completed successfully.

The next command lists computers in a particular domain:

```
C:\>net view /domain:corleone
```

Server Name	Remark
\\VITO	Make him an offer he can't refuse
\\MICHAEL	Nothing personal
\\SONNY	Badda bing badda boom
\\FREDO	I'm smart
\\CONNIE	Don't forget the cannoli

Again, `net view` requires access to NBNS across all networks that are to be enumerated, which means it typically only works against the local network segment. If NBNS is routed over TCP/IP, `net view` can enumerate Windows workgroups, domains, and hosts across an entire enterprise, laying bare the structure of the entire organization with a single unauthenticated query from any system plugged into a network jack lucky enough to get a DHCP address.

TIP

Remember that we can use information from ping sweeps (see Chapter 2) to substitute IP addresses for NetBIOS names of individual machines. IP addresses and NetBIOS names are mostly interchangeable; for example, `\\192.168.202.5` is equivalent to `\\SERVER_NAME`. For convenience, attackers often add the appropriate entries to their `%systemroot%\system32\drivers\etc\LMHOSTS` file, appended with the `#PRE` syntax, and then run `nbtstat -R` at a command line to reload the name table cache. They are then free to use the NetBIOS name in future attacks, and the name will be mapped transparently to the IP address specified in LMHOSTS.

Enumerating Windows Domain Controllers To dig a little deeper into the Windows network structure, we need to use a tool from the Reskit (microsoft.com/downloads/details.aspx?FamilyId=49AE8576-9BB9-4126-9761-BA8011FABF38&displaylang=en). In the next example, you'll see how the Reskit tool called `nltest` identifies the domain controllers in the domain we just enumerated using `net view` (domain controllers are the keepers of Windows network authentication credentials and are, therefore, primary targets of malicious hackers):

```
C:\>nltest /dclist:corleone
```

```
List of DCs in Domain corleone
    \\VITO (PDC)
    \\MICHAEL
    \\SONNY
```

The command completed successfully.

Netdom from the Reskit is another useful tool for enumerating key information about Windows domains on a wire, including domain membership and the identities of backup domain controllers (BDCs).

Enumerating Network Services with netviewx The `netviewx` tool by Jesper Lauritsen (see ibt.ku.dk/jesper/NTtools) works a lot like the `net view` command, but it adds the twist of listing servers with specific services. We often use `netviewx` to probe for the Remote Access Service (RAS) to get an idea of the number of dial-in servers that exist on a network, as shown in the following example (the `-D` syntax specifies the domain to enumerate, whereas the `-T` syntax specifies the type of machine or service to look for):

```
C:\>netviewx -D CORLEONE -T dialin_server
VITO,4,0,500, nt%workstation%server%domain_ctrl%time_source%dialin_server%
backup_browser%master_browser," Make him an offer he can't refuse "
```

The services running on this system are listed between the percent sign (%) characters. `netviewx` is also a good tool for choosing nondomain controller targets that may be poorly secured.

Dumping the NetBIOS Name Table with nbtstat and nbtscan `nbtstat` connects to discrete machines rather than enumerating the entire network. It calls up the NetBIOS name table from a remote system. The name table contains great information, as shown in the following example:

```
C:\>nbtstat -A 192.168.202.33
NetBIOS Remote Machine Name Table
```

Name	Type	Status
SERVR9	<00> UNIQUE	Registered
SERVR9	<20> UNIQUE	Registered
9DOMAN	<00> GROUP	Registered
9DOMAN	<1E> GROUP	Registered
SERVR9	<03> UNIQUE	Registered
INet Services	<1C> GROUP	Registered
IS SERVR9.....	<00> UNIQUE	Registered
9DOMAN	<1> UNIQUE	Registered
..__MSBROWSE__.	<01> GROUP	Registered
ADMINISTRATOR	<03> UNIQUE	Registered

MAC Address = 00-A0-CC-57-8C-8A

As illustrated, `nbtstat` extracts the system name (SERVR9), the domain it's in (9DOMAN), any logged-on users (ADMINISTRATOR), any services running (INet Services), and the network interface hardware Media Access Control (MAC) address. These entities can be identified by their NetBIOS service code (the two-digit number to the right of the name). These codes are partially listed in Table 3-2.

NetBIOS Code	Resource
<i>computer name</i> >[00]	Workstation Service
<i>domain name</i> >[00]	Domain name
<i>computer name</i> >[03]	Messenger Service (for messages sent to this computer)
<i>username</i> >[03]	Messenger Service (for messages sent to this user)
<i>computer name</i> >[20]	Server Service
<i>domain name</i> >[1D]	Master Browser
<i>domain name</i> >[1E]	Browser Service Elections
<i>domain name</i> >[1B]	Domain Master Browser

Table 3-2 Common NetBIOS Service Codes

The two main drawbacks to `nbtstat` are its restriction to operating on a single host at a time and its rather inscrutable output. Both of those issues are addressed by the free tool `nbtscan`, from Alla Bezroutchko, available at inetcat.net/software/nbtscan.html. `nbtscan` will “`nbtstat`” an entire network with blistering speed and format the output nicely:

```
C:\>nbtscan 192.168.234.0/24
Doing NET name scan for addresses from 192.168.234.0/24
IP address      NetBIOS Name   Server    User      MAC address
-----
192.168.234.36  WORKSTN12      <server>   RSMITH    00-00-86-16-47-d6
192.168.234.110 CORP-DC         <server>   CORP-DC    00-c0-4f-86-80-05
192.168.234.112 WORKSTN15      <server>   ADMIN      00-80-c7-0f-a5-6d
192.168.234.200 SERVR9         <server>   ADMIN      00-a0-cc-57-8c-8a
```

Coincidentally, `nbtscan` is a quick way to flush out hosts running Windows on a network. Try running it against your favorite Class C-sized network, and you’ll see what we mean.

Linux NetBIOS Enumeration Tools Although we’ve described a number of different Windows-based NetBIOS enumeration tools, an equal amount are available for Linux. One tool in particular is NMBscan by Grégoire Barbier (nmbscan.g76r.eu/). NMBscan provides the ability to enumerate NetBIOS by specifying different levels of verbosity:

```
nmbscan-1.2.4 # ./nmbscan
nmbscan version 1.2.4 - Sat Jul 19 17:41:03 GMT 2008
```

```
usage :
./nmbscan -L
-L show licence agreement (GPL)

./nmbscan {-d|-m|-a}
-d show all domains
-m show all domains with master browsers
-a show all domains, master browsers, and servers

./nmbscan {-h|-n} host1 [host2 [...]]
-h show information on hosts, known by ip name/address
-n show information on hosts, known by nmb name
```

We like to specify just the `-a` option to obtain a complete view of the NetBIOS network around us:

```
nmbscan-1.2.4 # ./nmbscan -a
nmbscan version 1.2.4 - Sat Jul 19 17:44:22 GMT 2008
domain EXAMPLE
  master-browser SLIPDIPDADOOKEN 10.219.1.201 -
  server SHARUCAN
    ip-address 10.219.1.20
    mac-address 01:18:F3:E9:04:7D
    ip-address 192.168.252.1
    ip-address 192.168.126.1
    server-software Windows Vista (TM) Ultimate 6.0
    operating-system Windows Vista (TM) Ultimate 6000
  server PIZZAZKICK
  server HADUCAN
    ip-address 10.219.1.207
    mac-address 00:0C:29:05:20:A7
    server-software Windows Server 2003 5.2
    operating-system Windows Server 2003 3790 Service Pack 2
  server GNA
  server SLIPDIPDADOOKEN
    ip-address 10.219.1.201
    mac-address 00:DE:AD:BE:EF:00
    ip-address 192.168.175.1
    ip-address 192.168.152.1
    server-software Windows 2000 LAN Manager
    operating-system Windows 5.1
domain -
  master-browser - 192.168.175.1 -
domain -
  master-browser - 192.168.152.1 -
```




Stopping NetBIOS Name Services Enumeration

All the preceding techniques operate over the NetBIOS Naming Service, UDP 137. If access to UDP 137 is restricted, either on individual hosts or by blocking the protocol at network routers, none of these activities will be successful. To prevent user data from appearing in NetBIOS name table dumps, disable the Alerter and Messenger Services on individual hosts. The startup behavior for these services can be configured through the Services Control Panel. On Windows 2000 and later, the Alerter and Messenger Services are disabled by default, plus you can disable NetBIOS over TCP/IP under the settings for individual network adapters. However, we've experienced unreliable success in blocking NBNS enumeration using the NetBIOS over TCP/IP setting, so we wouldn't rely on it (and, as you will see later in this chapter, there are many other misconceptions about this feature as well). Finally, be aware that if you block UDP 137 from traversing routers, you will disable Windows name resolution across those routers, breaking any applications that rely on NBNS.



NetBIOS Session Enumeration, TCP 139/445

<i>Popularity:</i>	8
<i>Simplicity:</i>	10
<i>Impact:</i>	8
<i>Risk Rating:</i>	9

Windows NT and its progeny have achieved a well-deserved reputation for giving away free information to remote pilferers. This reputation is almost singularly due to the vulnerability that we are going to discuss next—the Windows null session/anonymous connection attack.

Null Sessions: The Holy Grail of Enumeration If you've ever accessed a file or printed to a printer associated with a Windows machine across a network, chances are good that you've used Microsoft's Server Message Block (SMB) protocol, which forms the basis of Windows File and Print Sharing (the Linux implementation of SMB is called Samba). SMB is accessible via APIs that can return rich information about Windows—even to unauthenticated users. The quality of the information that can be gathered via this mechanism makes SMB one of the biggest Achilles' heels for Windows if not adequately protected.

To demonstrate the devastation that can arise from leaving SMB unprotected, let's perform some widely known hacking techniques that exploit the protocol. The first step in enumerating SMB is to connect to the service using the so-called "null session" command, shown next:

```
C:\>net use \\192.168.202.33\IPC$ "" /u:""
```

You might notice the similarity between this command and the standard `net use` syntax for mounting a network drive—in fact, they are nearly identical. The preceding

syntax connects to the hidden interprocess communications “share” (IPC\$) at IP address 192.168.202.33 as the built-in anonymous user (/u: "") with a null ("") password. If successful, the attacker now has an open channel over which to attempt the various techniques outlined in this section to pillage as much information as possible from the target, including network information, shares, users, groups, Registry keys, and so on. Regardless of whether you’ve heard it called the “Red Button” vulnerability, null session connections, or anonymous logon, it can be the single most devastating network foothold sought by intruders, as we will vividly demonstrate next.

NOTE

SMB enumeration is feasible over both TCP 139 (NetBIOS Session) and TCP 445 (SMB over raw TCP/IP, also called “Direct Host”). Both ports provide access to the same service (SMB), just over different transports.

Enumerating File Shares Some of the favorite targets of intruders are mis-ACL’d Windows file shares. With a null session established, we can enumerate the names of file shares quite easily using a number of techniques. For example, the built-in Windows `net view` command can be used to enumerate shares on remote systems:

```
C:\>net view \\vito
Shared resources at \\192.168.7.45
VITO
Share name      Type           Used as Comment
-----
NETLOGON        Disk           Logon server share
Test            Disk           Public access
The command completed successfully.
```

Two other good share-enumeration tools from the Windows Server 2003 Resource Kit are `srvcheck` and `srvinfo` (using the `-s` switch) (microsoft.com/downloads/details.aspx?familyid=9D467A69-57FF-4AE7-96EE-B18C4790CFFD&displaylang=en). `srvcheck` displays shares and authorized users, including hidden shares, but it requires privileged access to the remote system to enumerate users and hidden shares. `srvinfo`’s `-s` parameter lists shares along with a lot of other potentially revealing information.

One of the best tools for enumerating Windows file shares (and a whole lot more) is DumpSec (formerly DumpAcl), shown in Figure 3-3. It is available for free from SomarSoft (somalsoft.com). Few tools deserve their place in the NT security administrator’s toolbox more than DumpSec. It audits everything from file-system permissions to services available on remote systems. Basic user information can be obtained even over an innocuous null connection, and it can be run from the command line, making for easy automation and scripting. In Figure 3-3, we show DumpSec being used to dump share information from a remote computer.

Opening null connections and using the preceding tools manually is great for directed attacks, but most hackers commonly employ a NetBIOS scanner to check entire networks rapidly for exposed shares. Two tools that perform these tasks are SysInternals’s (acquired by Microsoft) ShareEnum (technet.microsoft.com/en-us/sysinternals/bb897442.aspx)

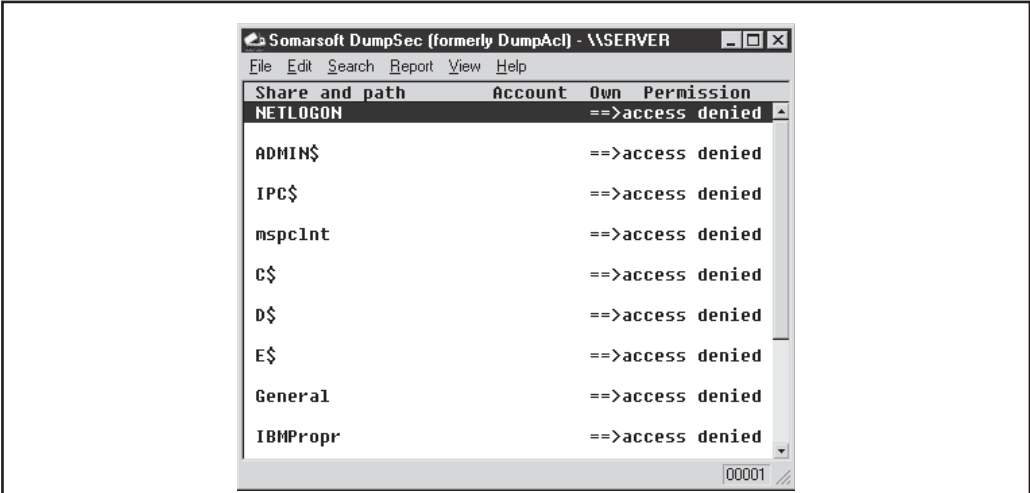


Figure 3-3 DumpSec reveals shares over a null session with the target computer.

and SoftPerfect's Network Scanner (softperfect.com/products/networkscanner/). ShareEnum has fewer configurable options, but, by default, it provides a good amount of information and has nice comparison features that may be useful for comparing results over time. SoftPerfect's Network Scanner is a bit more diverse but requires some minimal configuration beyond the default (see Figure 3-4).

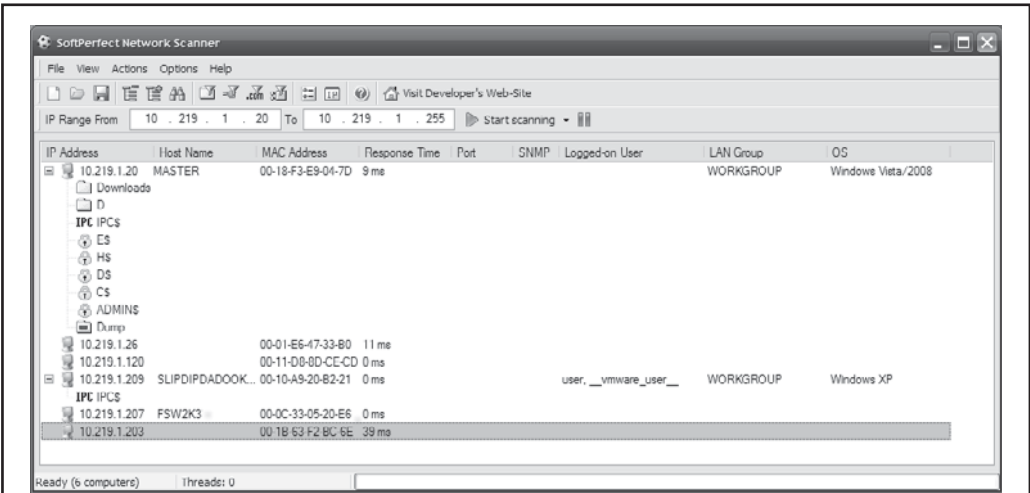


Figure 3-4 SoftPerfect's Network Scanner automatically scans subnets for open file shares.

Unlike older tools such as Legion, or the NetBIOS Auditing Tool (NAT), these newer tools target the “security professional” rather than the “hacker,” so unfortunately you are not likely to find password brute-forcing functionality included. Regardless, you can always use the older tools to do your dirty work, or use one of the brute-forcing tools mentioned later on in this book.

Legion can chew through a Class C IP network and reveal all available shares in its graphical interface. Version 2.1 includes a “brute-force tool” that tries to connect to a given share by using a list of passwords supplied by the user. For more on brute-force cracking of Windows, see Chapter 4. Another popular Windows share scanner is the NetBIOS Auditing Tool (NAT), based on code written by Andrew Tridgell. (NAT is available through the *Hacking Exposed* website, hackingexposed.com.) Neon Surge and Chameleon of the now-defunct Rhino9 Security Team wrote a graphical interface for NAT for the command-line challenged, as shown in Figure 3-5. NAT not only finds shares but also attempts forced entry utilizing user-defined username and password lists.

Registry Enumeration Another good mechanism for enumerating NT Family application information involves dumping the contents of the Windows Registry from the target. Most any application that is correctly installed on a given NT system leaves some sort of footprint in the Registry; it’s just a question of knowing where to look. Additionally, intruders can sift through reams of user- and configuration-related information if they gain access to the Registry. With patience, some tidbit of data that grants access can usually be found among its labyrinthine hives. Fortunately, Windows’s default configuration is to allow only administrators access to the Registry. Therefore, the techniques described next

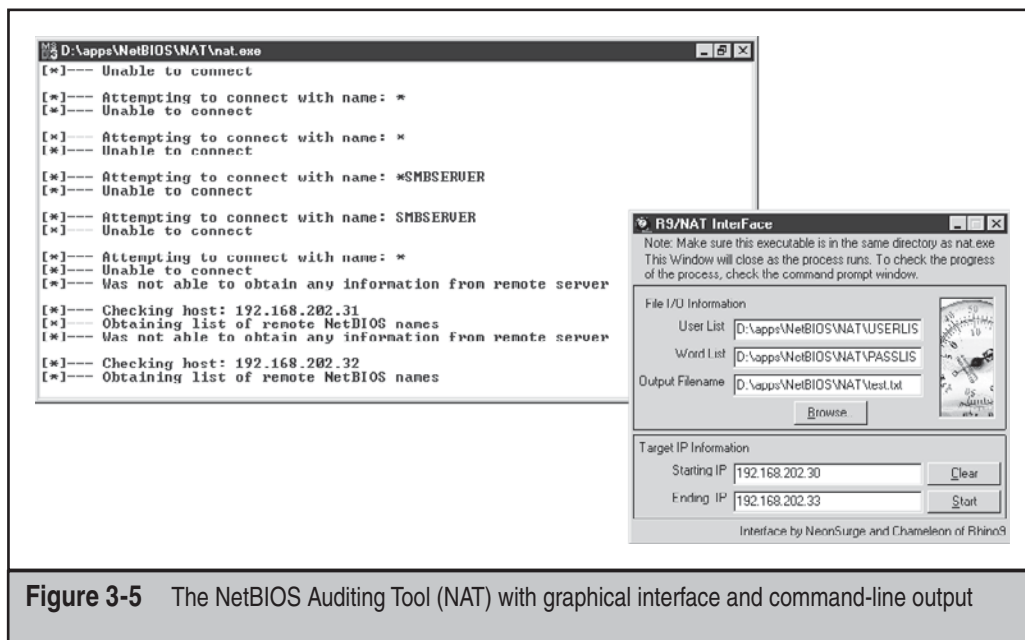


Figure 3-5 The NetBIOS Auditing Tool (NAT) with graphical interface and command-line output

will not typically work over anonymous null sessions. One exception to this is when the HKLM\System\CurrentControlSet\Control\SecurePipeServer\Winreg\AllowedPaths key specifies other keys to be accessible via null sessions. By default, it allows access to HKLM\Software\Microsoft\WindowsNT\Current Version.

If you want to check whether a remote Registry is locked down, the best tools are the `reg` (built into Windows XP, 2003, and later) and SomarSoft's DumpSec (once again). For pre-Windows 2003 systems, `regdmp` can be used instead of `reg` (`regdmp` was the original decommissioned tool; all of its functionality was then built into the `reg` utility). `reg /regdmp` is a rather raw utility that simply dumps the entire Registry (or individual keys specified at the command line) to the console. Although remote access to the Registry is usually restricted to administrators, nefarious do-nothings will probably try to enumerate various keys anyway in hopes of a lucky break. Hackers often plant pointers to backdoor utilities such as NetBus (see Chapter 4). Here, we check to see what applications start up with Windows:

```
C:\>reg query \\10.219.1.207\HKLM\SOFTWARE\MICROSOFT\
Windows\CurrentVersion\Run

! REG.EXE VERSION 3.0

HKEY_LOCAL_MACHINE\SOFTWARE\MICROSOFT\
Windows\CurrentVersion\Run

    VMware Tools REG_SZ
C:\Program Files\VMware\VMware Tools\VMwareTray.exe

    VMware User Process REG_SZ
C:\Program Files\VMware\VMware Tools\VMwareUser.exe

Adobe Reader Speed Launcher REG_SZ
"C:\Program Files\Adobe\Reader 8.0\Reader\Reader_sl.exe"

SunJavaUpdateSched REG_SZ
"C:\Program Files\Java\jre1.6.0_03\bin\jusched.exe"

HKEY_LOCAL_MACHINE\SOFTWARE\MICROSOFT\
Windows\CurrentVersion\Run\OptionalComponents
```

DumpSec produces much nicer output but basically achieves the same thing, as shown in Figure 3-6. The “Dump Services” report enumerates every Win32 service and kernel driver on the remote system, whether running or not (again, assuming proper access permissions). This information could provide a wealth of potential targets for attackers to choose from when planning an exploit. Remember that a null session is required for this activity.

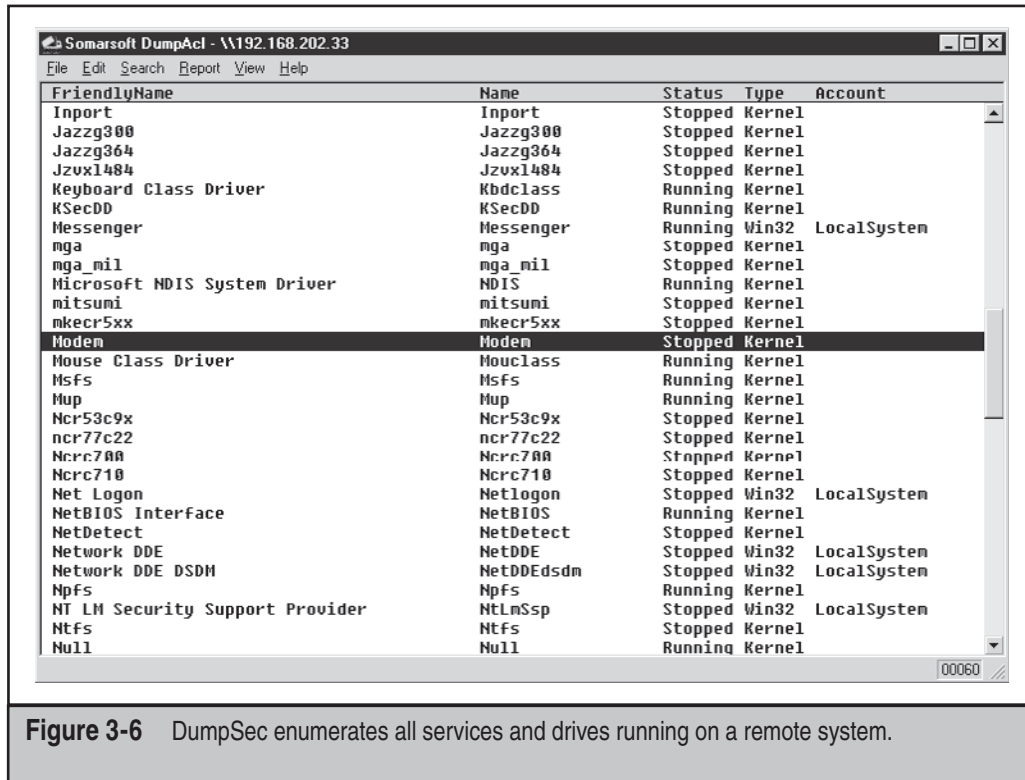


Figure 3-6 DumpSec enumerates all services and drives running on a remote system.

Enumerating Trusted Domains Remember the `nltest` tool, which we discussed earlier in the context of NetBIOS Name Service Enumeration? Once a null session is set up to one of the machines in the enumerated domain, the `nltest /server:<server_name>` and `/trusted_domains` syntax can be used to learn about further Windows domains related to the first. It's amazing how much more powerful these simple tools become when a null session is available.

Enumerating Users At this point, giving up share information probably seems pretty bad, but not the end of the world—at least attackers haven't been able to get at user account information, right? Wrong. Unfortunately, some Windows machines cough up user information over null sessions just about as easily as they reveal shares.

One of the most powerful tools for mining a null session for user information is, once again, DumpSec. It can pull a list of users, groups, and the NT system's policies and user rights. In the next example, we use DumpSec from the command line to generate a file containing user information from the remote computer (remember that DumpSec requires a null session with the target computer to operate):

```
C:\>dumpsec /computer=\\192.168.202.33 /rpt=useronly
/saveas=tsv /outfile=c:\temp\users.txt
```

```
C:\>cat c:\temp\users.txt
7/15/08 10:07 AM - Somarsoft DumpSec - \\192.168.202.33
UserName      FullName      Comment
Barzini       Enrico Barzini Rival mob chieftain
godfather     Vito Corleone  Capo
Godzilla      Administrator  Built-in account for administering the domain
Guest         Guest         Built-in account for guest access
lucca         Lucca Brazzi   Hit man
mike          Michael Corleone Son of Godfather
```

Using the DumpSec GUI, you can include many more information fields in the report, but the format just shown usually ferrets out troublemakers. For example, we once came across a server that stored the password for the renamed Administrator account in the Comments field!

Two other extremely powerful Windows enumeration tools are `sid2user` and `user2sid` by Evgenii Rudnyi (see evgenii.rudnyi.ru/soft/sid/sid.txt). These are command-line tools that look up NT Family SIDs from username input and vice versa. SID is the *security identifier*, a variable-length numeric value issued to an NT Family system at installation. For a good explanation of the structure and function of SIDs, read the excellent article at en.wikipedia.org/wiki/Security_Identifier. Once an intruder has learned a domain's SID through `user2sid`, that intruder can use known SID numbers to enumerate the corresponding usernames. Here's an example:

```
C:\>user2sid \\192.168.202.33 "domain users"
```

```
S-1-5-21-8915387-1645822062-1819828000-513
```

```
Number of subauthorities is 5
Domain is ACME
Length of SID in memory is 28 bytes
Type of SID is SidTypeGroup
```

Now we know the SID for the machine—the string of numbers beginning with S-1, separated by hyphens. The numeric string following the last hyphen is called the *relative identifier (RID)*, and it is predefined for built-in Windows users and groups such as Administrator and Guest. For example, the Administrator user's RID is always 500, and the Guest user's is 501. Armed with this tidbit, a hacker can use `sid2user` and the known SID string appended with an RID of 500 to find the name of the administrator's account (even if it has been renamed). Here's an example:

```
C:\>sid2user \\192.168.2.33 5 21 8915387 1645822062 18198280005 500
```

```
Name is godzilla
Domain is ACME
Type of SID is SidTypeUser
```

Note that S-1 and the hyphens are omitted. Another interesting factoid is that the first account created on any NT-based local system or domain is assigned an RID of 1000, and each subsequent object gets the next sequential number after that (1001, 1002, 1003, and so on—RIDs are not reused on the current installation). Therefore, once the SID is known, a hacker can basically enumerate every user and group on an NT Family system, past and present.

NOTE

sid2user/user2sid even works if RestrictAnonymous is set to 1 (defined shortly), as long as port 139 or 445 is accessible.

Here's a simple example of how to script user2sid/sid2user to loop through all the available user accounts on a system. Before running this script, we first determine the SID for the target system using user2sid over a null session, as shown previously. Recalling that the NT Family assigns new accounts an RID beginning with 1000, we then execute the following loop using the NT Family shell command FOR and the sid2user tool (see earlier) to enumerate up to 50 accounts on a target:

```
C:\>for /L %i IN (1000,1,1050) DO sid2user \\acmepdc1 5 21 1915163094
1258472701648912389 %i >> users.txt
C:\>cat users.txt
```

```
Name is IUSR_ACMEPDC1
Domain is ACME
Type of SID is SidTypeUser
```

```
Name is MTS Trusted Impersonators
Domain is ACME
Type of SID is SidTypeAlias
. . .
```

This raw output could be sanitized by piping it through a filter to leave just a list of usernames. Of course, the scripting environment is not limited to the NT shell—Perl, VBScript, or whatever is handy will do. As one last reminder before we move on, realize that this example will successfully dump users as long as TCP port 139 or 445 is open on the target, RestrictAnonymous = 1 notwithstanding.

NOTE

One of the myriad features of the all-encompassing Windows hacking suite, Cain and Abel (oxid.it/cain.html) is user enumeration. It even automates the process of first attempting the null session method described previously and then falls back to the sid2user method just described if the target's RestrictAnonymous is set to 1.

All-in-One Null Session Enumeration Tools Various developers have created a number of all-in-one null session enumeration tools so you can get the most bang for your buck with SMB enumeration. The tool that currently tops the list is Winfingerprint (sourceforge

.net/projects/winfingerprint). As suggested by all of the checkboxes viewable in Figure 3-7, Winfingerprint wins for overall functionality, as it has nearly everything you could hope for in a Windows enumeration tool, capable of enumerating everything mentioned previously and more. It can target a single host, lists or ranges of hosts, or just all visible hosts on a segment, and in addition to its null session functionality, Winfingerprint is also capable of enumerating Windows systems via Active Directory and WMI, making it a truly versatile Windows enumeration utility.

Another useful all-in-one tool is NBTEnum by Reed Arvin, although it can be more difficult to locate now that its website is no longer online (PacketStorm currently has it at packetstormsecurity.org/files/download/52547/NBTEnum33.zip). NBTEnum shines due to its extensive yet easy-to-read HTML output, intelligent brute-forcing capabilities, and its ability to enumerate a multitude of information using null sessions or under a particular user account. Using the tool is simple: to perform basic enumeration operations simply supply the `-q` option followed by the hostname. To enable intelligent brute forcing, use the `-s` option and include a dictionary file. NBTEnum (see Figure 3-8) first checks the server's account lockout policy and then attempts to brute force only a limited number of passwords so the limit is not reached.

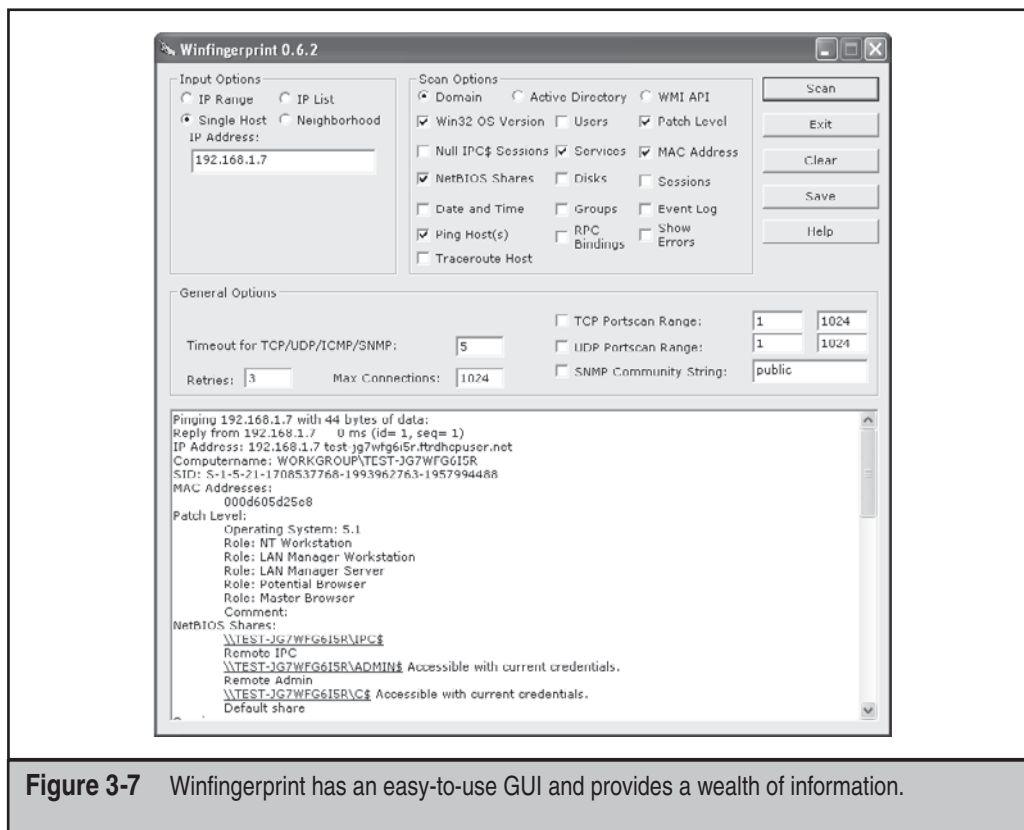


Figure 3-7 Winfingerprint has an easy-to-use GUI and provides a wealth of information.

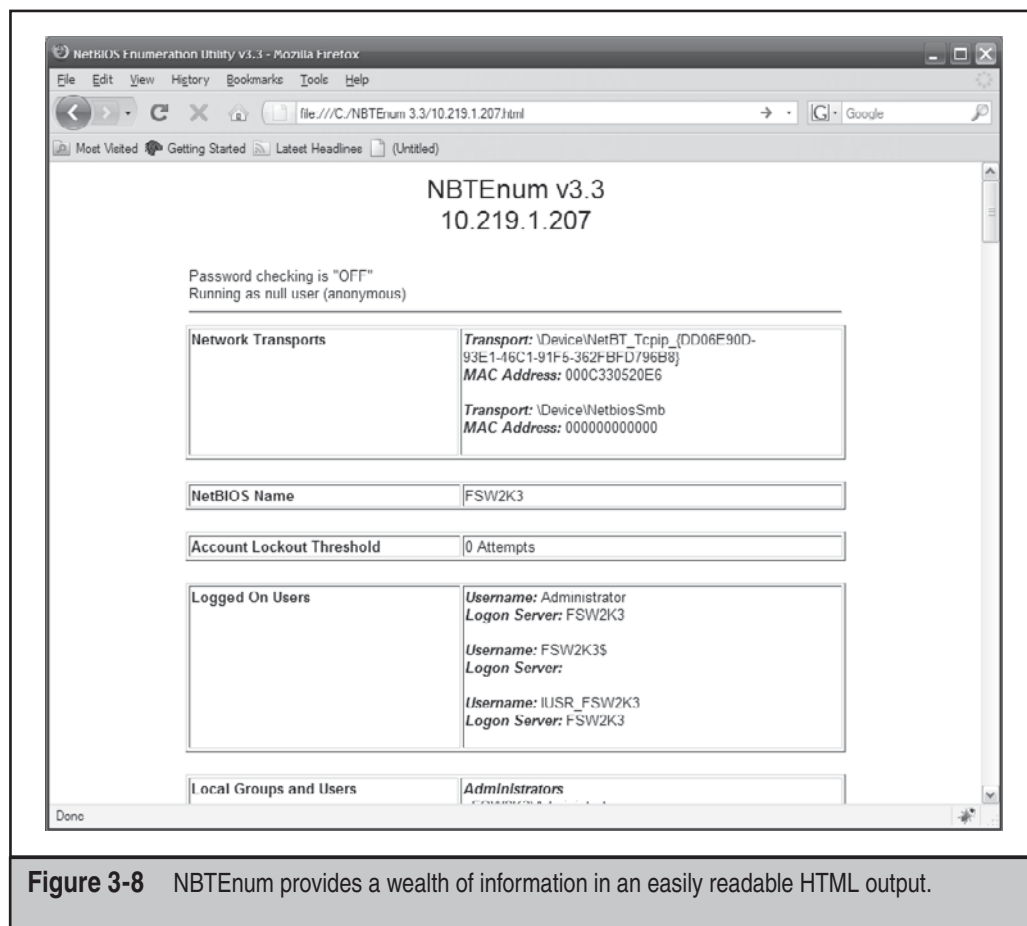


Figure 3-8 NBTEnum provides a wealth of information in an easily readable HTML output.

`enum`, developed by Razor Team from BindView (which has since been acquired by Symantec), is an excellent tool for SMB enumeration. Unfortunately, it is also older in comparison to Winfingerprint and can be much harder to find. It supports automatic setup and teardown of null sessions, password brute forcing, and a ton of additional features that make it a great addition to an attacker's toolkit. The following listing of the available command-line switches for this tool demonstrates how comprehensive it is:

```
C:\>enum
usage: enum [switches] [hostname|ip]
  -U: get userlist
  -M: get machine list
  -N: get namelist dump (different from -U|-M)
  -S: get sharelist
  -P: get password policy information
```

```
-G: get group and member list
-L: get LSA policy information
-D: dictionary crack, needs -u and -f
-d: be detailed, applies to -U and -S
-c: don't cancel sessions
-u: specify username to use (default " ")
-p: specify password to use (default " ")
-f: specify dictfile to use (wants -D)
```

Portcullis Security has developed a Linux clone of enum named enum4linux (labs.portcullis.co.uk/application/enum4linux/), which is a wrapper for common commands available within the Samba suite. It provides the same information plus a number of different options (edited for brevity):

```
enum4linux-0.7.0 # ./enum4linux.pl
Copyright (C) 2006 Mark Lowe (mrl@portcullis-security.com)
```

```
Usage: ./enum4linux.pl [options] ip
```

Options are (like "enum"):

```
-U          get userlist
-M          get machine list*
-N          get namelist dump (different from -U|-M)*
-S          get sharelist
-P          get password policy information*
-G          get group and member list
-L          get LSA policy information*
-D          dictionary crack, needs -u and -f*
-d          be detailed, applies to -U and -S*
-u username specify username to use (default "")
-p password specify password to use (default "")
-f filename specify dictfile to use (wants -D)*
```

* = Not implemented in this release.

Additional options:

```
-a          Do all simple enumeration (-U -S -G -r -o -n)
-h          Display this help message and exit
-r          enumerate users via RID cycling
-R range    RID ranges to enumerate
(default: 500-550,1000-1050, implies -r)
-s filename brute force guessing for share names
-k username User that exists on remote system
(default: administrator)
           Used to get sid with "lookupsid administrator"
```

```
-o          Get OS information
-w workgroup Specify workgroup manually (
usually found automatically)
-n          Do an nmblookup (similar to nbtstat)
-v          Verbose. Shows full commands being run
(net, rpcclient, etc.)
```

NetE is another older tool written by Sir Dystic of the Cult of the Dead Cow (cultdeadcow.com/tools/nete.html), but it works excellently and extracts a wealth of information from a null session connection. We like to use the /0 switch to perform all checks, but here's the command syntax for NetE to give you some idea of the comprehensive information it can retrieve via a null session:

C:\>**nete**

NetE v1.0 Questions, comments, etc. to sirdystic@cultdeadcow.com

Usage: NetE [Options] \\MachineNameOrIP

Options:

```
/0 - All NULL session operations
/A - All operations
/B - Get PDC name
/C - Connections
/D - Date and time
/E - Exports
/F - Files
/G - Groups
/I - Statistics
/J - Scheduled jobs
/K - Disks
/L - Local groups
/M - Machines
/N - Message names
/Q - Platform specific info
/P - Printer ports and info
/R - Replicated directories
/S - Sessions
/T - Transports
/U - Users
/V - Services
/W - RAS ports
/X - Uses
/Y - Remote registry trees
/Z - Trusted domains
```

Miscellaneous Null Session Enumeration Tools A few other NT Family enumeration tools bear mentioning here. Using a null session, `getmac` displays the MAC addresses and device names of network interface cards on remote machines. This output can yield useful network information to an attacker casing a system with multiple network interfaces. `getmac` works even if `RestrictAnonymous` is set to 1.

`Wininfo` by Arne Vidstrom at ntsecurity.nu extracts user accounts, shares, and interdomain, server, and workstation trust accounts. It'll even automate the creation of a null session if you want, by using the `-n` switch.



SMB Null Session Countermeasures

Null sessions require access to TCP 139 and/or 445 on Windows 2000 and greater, so the most prudent way to stop them is to filter TCP and UDP ports 139 and 445 at all perimeter network access devices. You could also disable SMB services entirely on individual NT hosts by unbinding WINS Client (TCP/IP) from the appropriate interface using the Network Control Panel's Bindings tab. Under Windows 2000 and later, this is accomplished by unbinding File and Print Sharing for Microsoft Networks from the appropriate adapter under Network and Dial-up Connections | Advanced | Advanced Settings.

Following NT 4 Service Pack 3, Microsoft provided a facility to prevent enumeration of sensitive information over null sessions without the radical surgery of unbinding SMB from network interfaces (although we still recommend doing that unless SMB services are necessary). It's called `RestrictAnonymous` after the Registry key that bears that name. Here are the steps to follow:

1. Open `regedt32` and navigate to `HKLM\SYSTEM\CurrentControlSet\Control\LSA`.
2. Choose Edit | Add Value and enter the following data:

Value Name:	RestrictAnonymous
Data Type:	REG_DWORD
Value:	1 (or 2 on Windows 2000 and later)

3. Exit the Registry Editor and restart the computer for the change to take effect.

On Windows 2000 and later, the fix is slightly easier to implement, thanks to Security Policies. The Security Policies MMC snap-in provides a graphical interface to the many arcane security-related Registry settings like `RestrictAnonymous` that needed to be configured manually under NT4. Even better, these settings can be applied at the Organizational Unit (OU), site, or domain level, so they can be inherited by all child objects in Active Directory if applied from a Windows 2000 and later domain controller. To do this, you must have the Group Policy snap-in. See Chapter 4 for more information about Group Policy.